CI/CD with Azure DevOps

`azure-pipelines.yml` (repo root) runs the Release steps of the runbook automatically:

Stage	Runs on	What it does
CI	every push to `main` and every pull request into `main`	Frontend: `npm ci`, `oxlint`, `vitest` (unit tests), `tsc -b && vite build`. Backend: `alembic upgrade head` on a throwaway Postgres 16, then the whole `pytest` suite, including the PostgreSQL RLS tests and the Redis tests. Test results appear on the run's **Tests** tab
Build	`main` only	`az acr build` for `fddt-backend` and `fddt-web`, tagged with the first 7 characters of the commit SHA
Deploy	`main` only, **after approval**	Prints the PITR target time and the previous image, runs the `fddt-migrate` job and waits for *Succeeded*, updates `fddt-api`, the three workers, `fddt-beat` and `fddt-web`, then checks `https://<APP_HOST>/api/health`

The pipeline **updates** existing apps. It does not create them, so do the first deployment with the runbook (steps 1–22). After that, every merge to `main` is released by the pipeline. You no longer need Cloud Shell for releases.

Nothing secret goes into Azure DevOps. The apps keep reading their secrets from Key Vault, and the pipeline signs in to Azure without a password, through workload identity federation.

Setting this up for the first time, with production already running? Follow the steps in order: 0 (check production and protect its data) → 1–6 (set up Azure DevOps) → 7 (the first release). The checklist at the end lists every step on one page. No step deletes data: the pipeline only **updates** the apps and runs migrations, which add to the database.

---

0. Before you start: production is already running

Do this once, in **Cloud Shell** (portal → the `>_` icon at the top), before touching Azure DevOps. It takes about 15 minutes. Run `source ~/fddt-deploy/reload.sh` first if you deployed with the runbook; otherwise set the two names by hand:

```
export RG=<your resource group>          # e.g. rg-fddtprod
export PG=<your PostgreSQL server name>  # e.g. fddtprod-pg
```

0.1 Check that production has the names the pipeline expects

The Deploy stage updates apps **by name**. List what exists:

```
az containerapp list -g $RG --query "[].name" -o tsv
az containerapp job list -g $RG --query "[].name" -o tsv
az acr list -g $RG --query "[].name" -o tsv
```

✅ You must see these apps: `fddt-api`, `fddt-worker-extraction`, `fddt-worker-vision`, `fddt-worker-forensics`, `fddt-beat`, `fddt-web`; the job `fddt-migrate`; and one registry (its name is your `ACR` value).

❌ **If any is missing or named differently, stop.** Production was not set up with the runbook (e.g. it is the single-app demo deployment) and the pipeline would fail at Deploy. Either rename the pipeline's app names in `azure-pipelines.yml` (Deploy stage) to yours, or move production to the runbook layout first. Ask before guessing: a wrong name fails the run, it does not damage anything.

0.2 Note which database version production is on

The migration job can print it without changing anything (`alembic current` only reads):

```
az containerapp job start -g $RG -n fddt-migrate --container-name migrate \
  --command "sh" --args "-c" "alembic current"
```

Wait 1–2 minutes, then open **portal › fddt-migrate › Execution history** → the newest execution → **Console logs** (logs arrive 2–5 minutes late). The last line is a revision id, e.g. `a2d6f9c3e7b4 (head)` . **Write it down.** The first pipeline release migrates from there to the newest version (step 7 lists what changes). If the log shows an error instead, fix that first (runbook step 13 lists the usual errors).

0.3 Take a backup you can name

Azure PostgreSQL already keeps automatic backups (point-in-time restore, 7+ days). Take an extra on-demand backup now, so there is a named restore point from before the first pipeline run:

```
az postgres flexible-server backup create -g $RG -n $PG --backup-name before-first-cicd
az postgres flexible-server backup list -g $RG -n $PG -o table      # the new backup is listed
```

If your CLI says the command does not exist, update it (`az upgrade`) or skip it: the automatic backups and the **PITR target** time the pipeline prints before migrating (step 7) are enough to restore from.

Check how long backups are kept: **portal › your PostgreSQL server › Settings › Backup and restore** → *Backup retention period*. Keep at least 7 days.

0.4 Check the admin settings the migration job uses

The migration job also runs `seed.py` . It never changes an existing admin account, but if `SEED_ADMIN_EMAIL` names an account that does **not** exist yet, it creates it, and outside local/test it now refuses a password shorter than 12 characters or the default `ChangeMe123!` . If you rotate secrets, keep `seed-admin-password` in Key Vault at least 12 characters long.

0.5 Rotate keys that were ever on a laptop

If Azure Storage, Document Intelligence or Azure OpenAI keys, or the JWT secret, were ever in a local `backend/.env` , rotate them in Azure and update Key Vault (runbook step 11) before go-live. Rotating the JWT secret signs everyone out once.

What you need to fill in

Collect these values before you start. Run the commands in Cloud Shell after `source ~/fddt-deploy/reload.sh` .

Value	Where it goes	How to get it
Subscription ID	Service connection	`az account show --query id -o tsv`
Subscription name	Service connection	`az account show --query name -o tsv`
Tenant ID	Service connection	`az account show --query tenantId -o tsv`

Value	Where it goes	How to get it
Client ID of the pipeline identity	Service connection (option B)	Created in step 2 below: `az identity show -g $RG -n $PREFIX-cicd --query clientId -o tsv`
`RG`	Variable group	`echo $RG` (e.g. `rg-fddtprod`)
`ACR`	Variable group	`echo $ACR`: the registry **name** (e.g. `fddtprodacr`), not the `.azurecr.io` address
`APP_HOST`	Variable group	The address users open, **without** `https://`. With a custom domain: `echo $APP_DOMAIN`. Sandbox without one: `az containerapp show -g $RG -n fddt-web --query properties.configuration.ingress.fqdn -o tsv`

---

1. Project, repository and agents

1. In **https://dev.azure.com/<org>**, create a project (or use the client's), then **Repos › Files › Import repository**, or push to the empty repo:

   ```
   git remote add azure https://dev.azure.com/<org>/<project>/_git/FDDT
   git push azure main
   ```

2. **Check that you can run builds.** Go to **Organization settings › Pipelines › Parallel jobs**. *Microsoft-hosted* must show at least **1**. New organizations often show **0**. In that case:
   - request the free grant at **https://aka.ms/azpipelines-parallelism-request** (takes 2–3 business days), or
   - buy one parallel job (**Organization settings › Billing**, ~$40/month), or
   - run a self-hosted Linux agent with Docker installed and set `pool: name: <your pool>` in the YAML.

   Without one of these, every run fails with *No hosted parallelism has been purchased or granted*.

2. Service connection `fddt-azure`

The pipeline signs in to Azure with this connection. The name **must be exactly** `fddt-azure`. Otherwise, change `serviceConnection` at the top of `azure-pipelines.yml`.

The identity needs **Contributor on the resource group** and nothing else. Contributor covers `az acr build`, `az containerapp update` and starting the migration job. It does not need Key Vault access.

Option A: automatic (needs rights in Entra ID)

Project settings › Service connections › New service connection › Azure Resource Manager → *Identity type*: **App registration (automatic)** → *Credential*: **Workload identity federation** → *Scope level*: **Subscription** → pick the subscription and the resource group `rg-fddtprod` → *Service connection name*: `fddt-azure` → leave *Grant access permission to all pipelines* unticked → **Save**.

This fails if your account cannot create app registrations in the client's Entra ID, or cannot assign roles on the resource group. Sandbox accounts usually can't do either. If it fails, use option B.

Option B: managed identity (works in most sandboxes)

The client's admin, or anyone with **Owner** or **User Access Administrator** on the resource group, runs this in Cloud Shell:

```
source ~/fddt-deploy/reload.sh
az identity create -g $RG -n $PREFIX-cicd -o none
CICD_CLIENT_ID=$(az identity show -g $RG -n $PREFIX-cicd --query clientId -o tsv)
CICD_PRINCIPAL=$(az identity show -g $RG -n $PREFIX-cicd --query principalId -o tsv)
az role assignment create --assignee-object-id $CICD_PRINCIPAL --assignee-principal-type ServicePrincipal \
  --role Contributor --scope $(az group show -n $RG --query id -o tsv)
echo "Client ID: $CICD_CLIENT_ID"
echo "Tenant ID: $(az account show --query tenantId -o tsv)"
echo "Subscription: $(az account show --query id -o tsv)  $(az account show --query name -o tsv)"
```

Then in Azure DevOps:

1. **Project settings › Service connections › New service connection › Azure Resource Manager** → *Identity type*: **Managed identity** (on older UIs: **Workload identity federation (manual)**) → **Next**.
2. *Service connection name*: `fddt-azure`. The next page shows an **Issuer** and a **Subject identifier**. Copy both and leave the page open.
3. Back in Cloud Shell, create the federated credential on the identity with those two values:

   ```
   az identity federated-credential create -g $RG --identity-name $PREFIX-cicd -n azure-devops \
     --issuer "<Issuer from Azure DevOps>" --subject "<Subject identifier from Azure DevOps>" \
     --audiences api://AzureADTokenExchange
   ```

4. Fill in the rest of the form: *Subscription ID*, *Subscription name*, *Tenant ID*, and **Client ID** = the identity's client ID from above. Tick **Verify** if it's offered, then **Save**.

✅ **Verify:** open the connection. *Edit › Verify* says *Verification succeeded*. If it says *AADSTS70021: No matching federated identity record*, the issuer or subject was copied wrong. Delete and re-create the federated credential.

3. Variable group `fddt-production`

Pipelines › Library › + Variable group → name `fddt-production` → add:

Name	Example
`RG`	`rg-fddtprod`
`ACR`	`fddtprodacr`
`APP_HOST`	`fddt.client.com` or `fddt-web.<hash>.uaenorth.azurecontainerapps.io`

None of these is secret, so don't lock them. **Save**.

4. Environment `fddt-production` (approval gate)

Pipelines › Environments › New environment → name `fddt-production` → *Resource*: **None** → **Create**. Then ⋮ › **Approvals and checks**:

- **+ › Approvals**: add the people who approve releases. Only they can let the Deploy stage start.
- **+ › Exclusive lock**: two releases never run at the same time. Set *Lock behavior* to **sequential**.

5. Create the pipeline

Pipelines › New pipeline › Azure Repos Git → the FDDT repo → **Existing Azure Pipelines YAML file** → branch `main`, path `/azure-pipelines.yml` → **Run**.

On the first run, Azure DevOps pauses and asks to **Permit** the pipeline to use the service connection, the variable group and the environment. Click **View › Permit** for each. It asks only once.

6. Run CI on pull requests

Azure Repos ignores the `pr:` block in the YAML. Pull requests are validated by a branch policy instead: **Repos › Branches** › `main` › ⋮ › **Branch policies › Build validation › +** → pick this pipeline → *Trigger*: Automatic, *Policy requirement*: **Required** → **Save**. A PR can then merge only if CI passed. Build and Deploy are skipped for PRs.

7. The first release through the pipeline

1. **Pick a quiet time** (evening or weekend). This release signs everyone out once and briefly locks the documents table (below).
2. Push `main` to the Azure DevOps repo (step 1) or click **Run pipeline**. Wait for **CI** and **Build** to go green. Nothing in production has changed yet.
3. The approvers get an email. Before approving, check that the time is right and step 0 is done. **Review › Approve**.
4. Watch the **Deploy** log. In order it prints:
   - `PITR target (UTC): …`: **write this time down.** It is the moment to restore to if anything goes wrong.
   - `Previous API image: …`: write this down too (the image to go back to).
   - The migration execution name, then `Running` … `Succeeded`. If it says `Failed`, the apps were **not** updated and production keeps running the old version: see Troubleshooting.
   - `Deployed <tag>`, then `Healthy` from the smoke test.
5. **Check the result** (5 minutes):
   - Sign in (everyone has to sign in again once, expected).
   - Open the Case Queue: old cases are all there, 50 per page with Previous / Next.
   - Open an existing case: its documents and PDF preview load.
   - Upload a test PDF: it processes, and its checks include **Analysis integrity**.
   - Platform admin › Companies › Edit: the **Max pages per PDF** field shows 100.
   - Confirm the database moved to the new version: run step 0.2 again; the last log line is `a9c4e2f7b1d8 (head)`.

What this release's migrations change

Production on `a2d6f9c3e7b4` gets these six, in this order. **None deletes data**: each adds columns, a table or rows, or moves rows.

Migration	Change	What you notice
`b3e6d9a2f4c7`	Platform-admin audit rows become platform-only (`company_id` set to NULL, the company kept inside the row)	Companies no longer see platform-admin actions in their Audit History
`c6f1a8e3d9b5`	`cases.deleted_at`, `cases.deleted_by_user_id` (soft delete)	Nothing

Migration	Change	What you notice
`d7a2c4e9f1b6`	`refresh_tokens` table, `users.token_version`	**Everyone is signed out once**
`e1f4b7a2c9d3`	`documents.file_size_bytes` INTEGER → BIGINT	Values unchanged. The documents table is **locked while it is rewritten** (seconds for thousands of rows; uploads wait meanwhile)
`f5a8c3e1b7d2`	`companies.max_file_pages`, 100 for existing companies	PDFs over 100 pages are rejected
`a9c4e2f7b1d8`	Check type `analysis_integrity` + rules `integrity.prompt_injection` (60) and `integrity.analysis_incomplete` (30) for every company	Cases scored from now on: a document that addresses the AI → high; a document whose extraction failed → medium (was low). Existing scores do not change

If step 0.2 showed a revision **other than** `a2d6f9c3e7b4`, the list is longer or shorter. Run `alembic history` locally to see every migration between that revision and the newest one.

If something goes wrong

Situation	What to do	Data
Migration `Failed`	Nothing to roll back: the apps were not updated. Read the job's console logs, fix, run the pipeline again	Untouched (a failed migration is rolled back by PostgreSQL)
Migrated, but the new version misbehaves	Fix forward if you can (a new commit). Otherwise restore the database to the **PITR target** time and redeploy the **previous image**: runbook Rollback	Anything written **after** the PITR time is lost (uploads, reviews), so only restore if the release is unusable
Apps fine, smoke test failed	Usually `APP_HOST` is wrong (Troubleshooting). The release itself is live	Untouched

Don't rerun an older pipeline run after a migration: the old code does not know the new columns.

Day-to-day

- **Release:** merge to `main` → CI → Build → the approvers get an email → **Review › Approve** → Deploy.
- **Check a release:** the Deploy log prints the migration execution name, each status poll, `Deployed <tag>` and the `/api/health` answer. In the portal, each app's **Application › Revisions and replicas** shows the new revision.
- **Rollback without a migration:** open the last good run → **Deploy** stage → **Rerun stage**. That run's commit is redeployed (its images are still in ACR).
- **Rollback when the release had a migration:** don't rerun an older Deploy. The old image can't run `alembic upgrade head` against a newer database. Follow the runbook's Rollback. The Deploy log shows the **PITR target** time.
- **Docs-only commits** (`docs/**`, `*.md`) don't trigger the pipeline.

Troubleshooting

Error	Cause / fix
No hosted parallelism has been purchased or granted	See step 1.2
The pipeline is not valid. … service connection fddt-azure could not be found	The connection is named differently, or the pipeline was not permitted to use it (step 5)
AuthorizationFailed … does not have authorization to perform action	The Contributor role is missing on the RG, or was assigned less than ~5 minutes ago
`az acr build` : *The resource with name '…' … could not be found*	`ACR` holds the login server instead of the registry name, or the wrong `RG`
Migration `Failed`	The apps were not touched. Open **fddt-migrate › Execution history** → the execution → **Console logs**. Runbook step 13 lists the usual errors
Smoke test fails but the apps are *Running*	`APP_HOST` is wrong (it must have no `https://`), or the custom domain/TLS isn't bound yet: use the `*.azurecontainerapps.io` FQDN
CI `pytest` fails only in the pipeline	Run the same suite locally against Postgres and Redis: local-setup.md › Tests
CI `vitest` fails	A frontend unit test failed. Run `npm test` in `frontend/` locally
Migration log: *seed.py: refusing to set the platform admin password*	`SEED_ADMIN_EMAIL` names an account that doesn't exist, and `seed-admin-password` is shorter than 12 characters or the default. Set a strong value in Key Vault, or set `SEED_ADMIN_EMAIL` to the existing admin (step 0.4)
Apps start but crash with *Permission denied* writing a file	The image now runs as a non-root user (uid 10001). The app writes only to `/tmp` ; a custom volume mount or start script that writes elsewhere needs that path to be writable

First-time checklist

#	Step	Done
0.1	Production has `fddt-api` , the 3 workers, `fddt-beat` , `fddt-web` , job `fddt-migrate` , a registry	☐
0.2	Current database revision written down: `__________`	☐
0.3	On-demand backup `before-first-cicd` taken; retention ≥ 7 days	☐
0.4	`seed-admin-password` ≥ 12 characters	☐
0.5	Keys that were ever on a laptop rotated	☐
1	Azure DevOps project, repo imported, ≥ 1 parallel job	☐

#	Step	Done
2	Service connection `fddt-azure` verified	☐
3	Variable group `fddt-production`: `RG`, `ACR`, `APP_HOST`	☐
4	Environment `fddt-production` with approvers + exclusive lock	☐
5	Pipeline created from `/azure-pipelines.yml`, permissions granted	☐
6	Build validation policy on `main`	☐
7	First release: PITR target `_____`, previous image `_____`, smoke checks passed, revision `a9c4e2f7b1d8`	☐